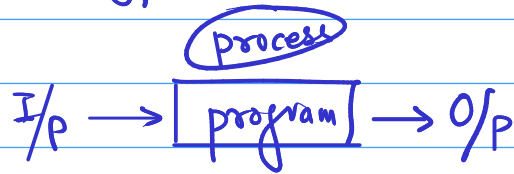


★ Data types, constants and variables

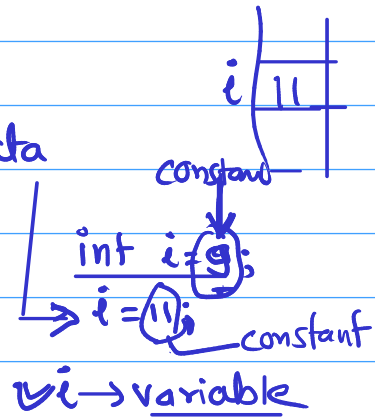
⇒



→ requires memory to store data

→ some data is constant

→ some data is variable



→ `char ch = 'x';`

Annotations:

- An arrow points from the text "constant" to the single quote character `'`.
- An arrow points from the text "variable" to the variable `ch`.

`ch + 9 + 'z' + ch`

Annotations:

- An arrow points from the text "constant" to the `9`.
- An arrow points from the text "constant" to the single quote character `'`.

⇒ Data type → constant & variable have data types

→ integral values (without floating point)

→ real value

→ char values

→ void

valid english?
here you there no why is ten.

What is C language? → C standard

Is Compiler a C language?

C language prog → Compiler → Binary code
machine language prog.

⇒ Integer data types

int x = 7;
int y = 290;

[x 7] how much memory?
[y 290] how is 7 stored?
decimal → Binary

→ Size of int ≥ 2 bytes

→ Different compilers have different size of int

→ Most compiler - 4 bytes.

1 byte = 8 bits

1		2
2		4
3		8
4		16
8		2 ⁸

4 bytes
-2³¹

int num = ?
min val

4 bytes
max val
2³¹ - 1

4 bytes = 4 × 8 bits = 32 bit 2³²

-2³¹ + 2³¹ - 1

n-bits - signed ← positive negative

→ int is signed by default.

-2ⁿ⁻¹ to 2ⁿ⁻¹ - 1

→ If you try to store numbers outside this range then it is called overflow.
min 0

→ unsigned int x = ?

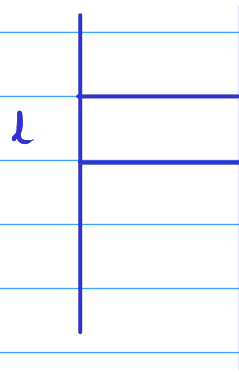
→ You should only store (+)ve values
max 2³² - 1 (4 byte)

DISCLAIMER: Never mix signed and unsigned variables in an expression.

→ prefer signed int over unsigned int.
long long

2³²

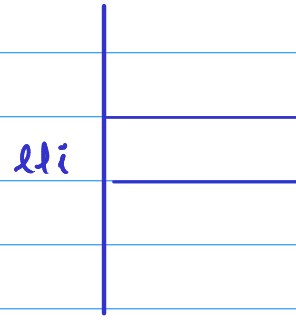
Same {
int long l;
long int l;
long l;
→ can hold (+)ve and (-)ve values.



→ size depends on compiler
→ size will be greater than or equal to size of int.

{
unsigned long ul;
unsigned long int ul;
long unsigned ul;

long long int lli;



→ depends on compiler
→ size ≥ long

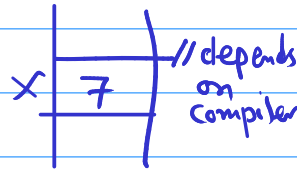
Size

1 to 100

short ≤ int ≤ long ≤ long long
↑
minimum 2 bytes

int long long → signed/unsigned
default

Short int x = 7;

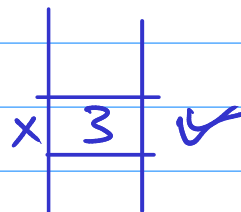


// depends on compiler

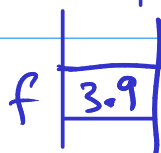
→ How to know size of int, long, long long?

→ Real values (floating point values — with fractional part).

int x = 3.9;
ignore



float f = 3.9;



// size is 4 bytes.

double j = 999.8762;



// size is 8 bytes

→ double can store very large and very small numbers.

→ double is better at precision than float.

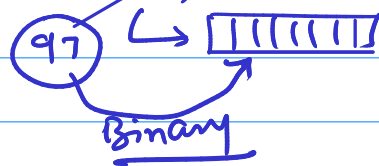
long double ld; → even more range

→ There is no long float.

→ What is its size? → Find on your own.

characters

char ch = 'a';



ASCII → American standard code for Information Interchange.

A - 65
B - 66
a - 97
b - 98

char ch = 'ab'; // Invalid statement

└┘
one character.

int x = 7.3;
└┘ └┘
variable constant
int double

→ By default

integer constant — int
floating-point constant — double

int x = 7.3f;
└┘
constant
float

int x = 95L;
└┘
long

int x; // declaration — garbage — random
int x=9; // declaration and initialization

x	10
y	15

scan

print

int x; scanf("%d", &x);
int y=7; scanf("%d", &y);

printf("%d", x);
printf("%d", y);

long z=991; scanf("%ld", &z);

printf("%ld", z);

short p=7; scanf("%hd", &p);

printf("%hd", p);

long long q; scanf("%lld", &q);

printf("%lld", q);

float g; scanf("%f", &g);

printf("%f", g);

double k; scanf("%lf", &k);

printf("%lf", k);

long double m; ?

?

char ch; scanf("%c", &ch);

printf("%c", ch);